

PROJECT REPORT

Terminal Quiz & Leaderboard Engine

Java based terminal quiz application

1. Introduction

This project is a small quiz program written in Java. It runs in the terminal, so the user interacts with it by typing in choices instead of using a GUI. A player enters a name, answers the questions one by one, and gets the score at the end. The score is then added to a simple leaderboard.

I kept the project simple because the main goal was to practise Java programming rather than build a large application. The project mainly uses classes, ArrayList, loops, Scanner, file handling and a little exception handling.

The questions are kept in questions.txt and the scores are kept in leaderboard.txt. This also means the program can be closed and opened again without losing the saved leaderboard.

2. Problem Statement

If the quiz were only a list of questions, it would be easy. The harder part is handling everything around it: taking input, checking whether the input is valid, deciding whether an answer is correct, keeping the score and saving the result.

The program handles these jobs by splitting them between a few classes. Question stores the data for one question, QuizManager handles most of the quiz and file work, and Main deals with the menu and user input. This keeps the program easier to follow than putting everything into one file.

3. Functional Requirements

No.	Requirement	What the program does
FR1	Start the application	Shows the quiz menu after loading the questions.
FR2	Take a quiz	Asks the player to enter a name and answer all available questions.
FR3	Validate answers	Accepts only option numbers 1 to 4.
FR4	Check answers	Shows whether the answer is correct and updates the score.
FR5	Show result	Displays score and percentage after the quiz.
FR6	Save score	Writes the player name and score into leaderboard.txt.
FR7	Show leaderboard	Reads saved scores and displays them in descending order.
FR8	Show questions	Displays the current question bank.
FR9	Handle missing question file	Creates a small sample file if questions.txt is missing.
FR10	Run tests	TestQuizEngine checks the main parts of the project.

4. Non-functional Requirements

- The program should be easy to start on any computer with a Java JDK installed.
- The implementation should stay reasonably easy to read and understand. It uses basic Java features instead of advanced libraries.
- The application should stay small and lightweight. It does not need a database, server or external Java library.
- Common input mistakes should not immediately stop the program. Invalid menu choices and invalid quiz answers are handled and the user is asked again.
- Scores should remain available after the program is closed, which is why they are written to a file.

5. System Architecture

The project has three main Java classes and two text files. Main.java starts the program and shows the menu. QuizManager does most of the work, such as loading questions, running the quiz and saving scores. Question.java is used to store the data for a single question. TestQuizEngine.java is kept separate for basic testing.

System Architecture

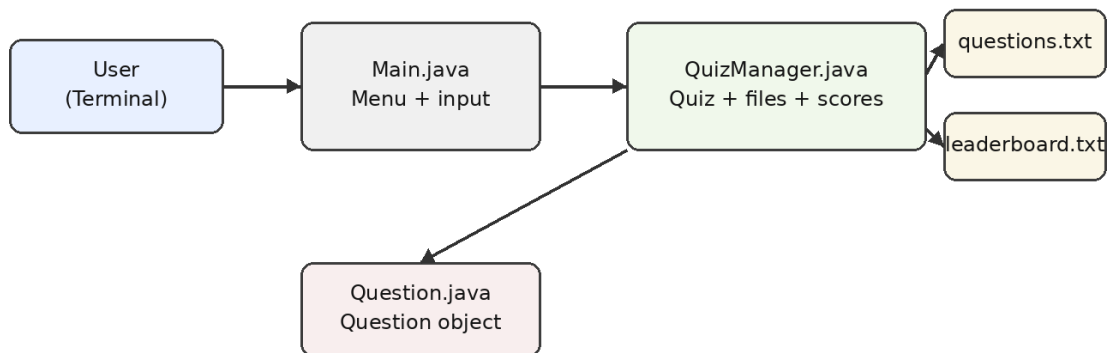


Figure 1. Simple system architecture used in the project.

6. Design Diagrams

There is one main actor here: the user. From the menu, the user can take a quiz, view the leaderboard, view the question bank or exit the program.

Use Case Diagram

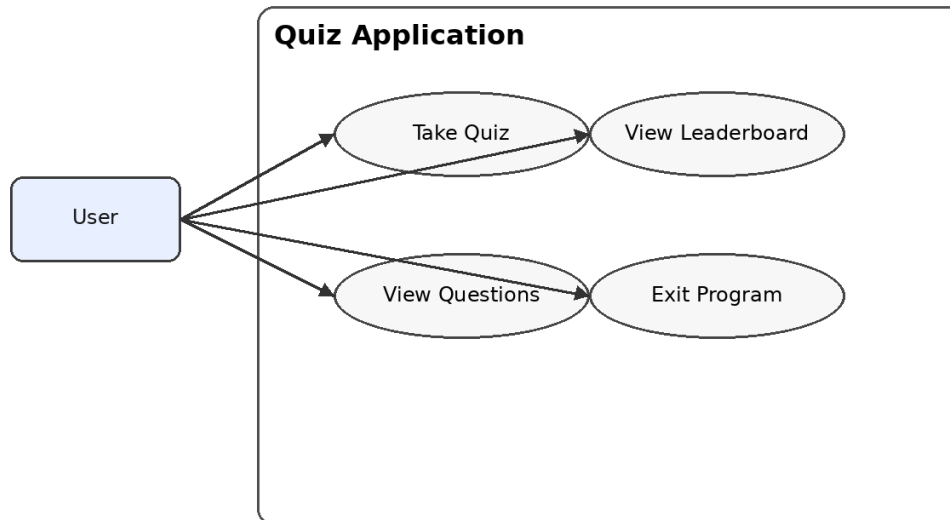


Figure 2. Use case diagram.

The normal flow is quite straightforward. The program loads the questions first, then shows the menu. When the user selects Take Quiz, the questions are copied and shuffled. They are then shown one at a time. After the last question, the score is calculated and saved to the leaderboard.

Workflow Diagram

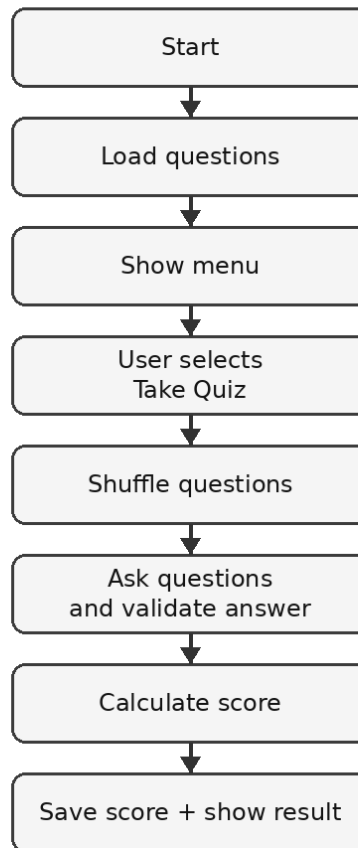


Figure 3. Main quiz workflow.

The sequence below follows one quiz attempt. Main receives the user choice and calls QuizManager. QuizManager picks a question, prints it, reads the answer and checks it. After all questions are done, QuizManager returns the score and Main saves it.

Sequence Diagram - Taking a Quiz

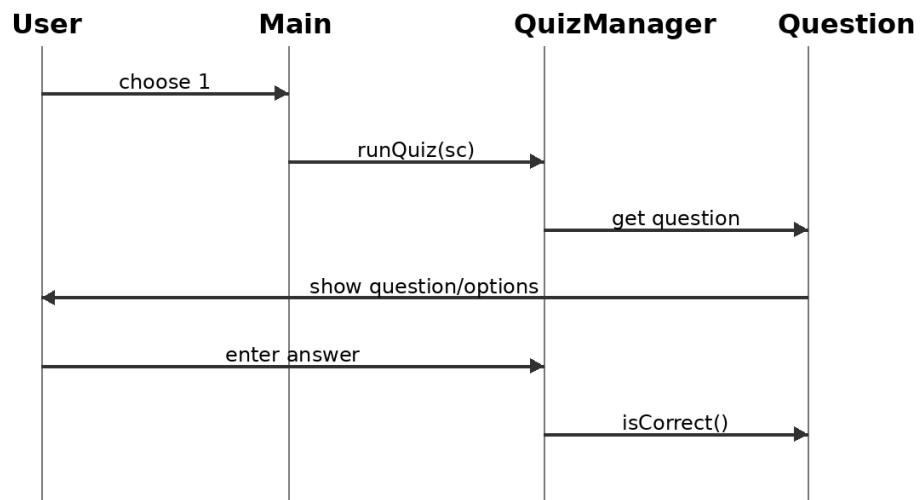


Figure 4. Quiz sequence diagram.

Question is mainly used for storing question data. QuizManager contains most of the actual quiz and file logic, while Main connects the menu and user input to QuizManager. TestQuizEngine is separate because it is only used for testing.

Class / Component Diagram

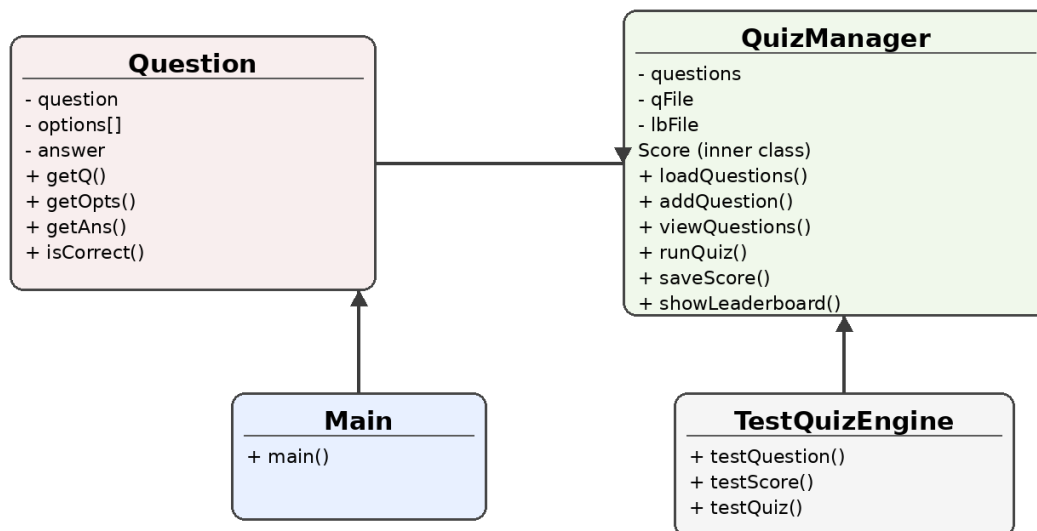
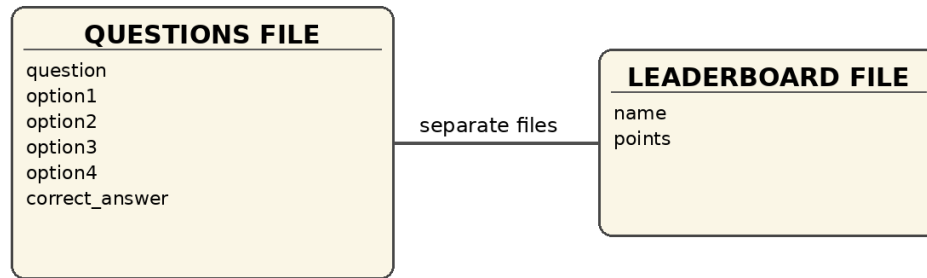


Figure 5. Class/component view.

This project does not use a relational database, so a normal ER diagram would be a little misleading. Instead, the diagram shows the two files used for storage: questions.txt for the question bank and leaderboard.txt for saved scores.

ER Diagram - File Based Storage



This is a storage view, not a relational database ER model, because the project uses text files.

Figure 6. File-based storage view.

7. Design Decisions & Rationale

I made these choices mainly because they were enough for the size of this project and kept the code easier for me to understand while building it.

Decision	Reason
Terminal interface	A terminal interface was enough for this project and let me focus on the Java logic instead of spending time building a GUI.
Text files instead of database	The amount of data is very small, so text files are enough for this version. A database would be unnecessary here.
Separate Question class	Keeping question data in its own class makes the rest of the code easier to read.
QuizManager for main logic	Most of the quiz, file and leaderboard work is kept in one place so Main.java does not become too large.
Collections.shuffle()	Shuffling a copied list changes the order for the quiz without changing the original question list.
Manual score sorting	A nested loop is enough for a small leaderboard and is also easier to understand at this stage of the project.
Basic tests instead of JUnit	I kept the tests simple so they can be run directly from the terminal without adding JUnit or another library.

8. Implementation Details

Main.java is the starting point of the program. It creates Scanner and QuizManager, loads the questions and keeps showing the menu until the user selects Exit. It also asks for the player name. If nothing is entered, the program uses "Player".

```
choice = Integer.parseInt(sc.nextLine());
```

Question.java is a small class that stores the question text, four options and the correct option number. The isCorrect() method is intentionally simple: it just compares the selected option with the stored answer.

```
return choice == answer;
```

QuizManager reads questions.txt line by line. Each line is split using the pipe character, then the question, four options and correct answer are used to create a Question object. Blank or badly formatted lines are skipped.

Before starting a quiz, the program makes a copy of the question list and shuffles that copy. This was done so the order changes between attempts without changing the original question bank.

```
ArrayList<Question> quiz = new ArrayList<Question>(questions);  
Collections.shuffle(quiz);
```

For each question, the program keeps asking until the user enters 1, 2, 3 or 4. After the last question, it prints the score and percentage.

For the leaderboard, the old scores are read first. The new score is added, the list is sorted from highest to lowest, and then the whole file is written again. It is a simple approach, but it is enough for the small amount of data used here.

The test class has three small tests. One checks Question.isCorrect(). Another checks that a question can be added and a score can be saved. The last test runs a one-question quiz using Scanner with a prepared input string. It is basic testing, but it covers some important parts of the program.

9. Screenshots / Results

```
===== Quiz App =====  
1. Take Quiz  
2. View Leaderboard  
3. View Questions  
4. Exit  
Enter choice:
```

```
===== Quiz App =====
```

- 1. Take Quiz
- 2. View Leaderboard
- 3. View Questions
- 4. Exit

Enter choice: 1

Enter your name: D

Question 1 of 49

In Round Robin scheduling, decreasing the time quantum generally makes the scheduling behavior approach?

- 1. FCFS
- 2. SJF
- 3. Priority scheduling
- 4. Non-preemptive scheduling

Your answer: |

Question 1 of 49

In Round Robin scheduling, decreasing the time quantum generally makes the scheduling behavior approach?

- 1. FCFS
- 2. SJF
- 3. Priority scheduling
- 4. Non-preemptive scheduling

Your answer: 3

Wrong. Correct answer: 2

```
===== Result =====  
Score: 14/49  
Percentage: 28.571428571428573%  
Your score has been saved.
```

```

===== Quiz App =====
1. Take Quiz
2. View Leaderboard
3. View Questions
4. Exit
Enter choice: 2

===== Leaderboard =====
1. D - 14

```

10. Testing Approach

I used both manual testing and the TestQuizEngine class. Manual testing was useful for checking the complete flow because it shows what an actual user sees. The test class was useful for checking a few smaller pieces of logic without having to go through the whole menu every time.

Test	Input / Action	Expected result
Open program	Run Main	Menu appears
Invalid menu	Enter a value such as 7	Program asks for 1 to 4
Blank name	Press Enter at name input	Name becomes Player
Invalid quiz answer	Enter text or a number outside 1-4	Program asks again
Correct answer	Enter the correct option	Score increases by 1
View leaderboard	Choose option 2	Saved scores appear
View questions	Choose option 3	Question bank is displayed

Test method	What is checked	Result
testQuestion()	isCorrect() returns true for the right option and false for the wrong one.	Pass
testScore()	Question can be added and a score can be saved.	Pass
testQuiz()	A one-question quiz returns a score of 1 when the answer is 2.	Pass

11. Challenges Faced

- Handling terminal input was one of the first issues because menu input and quiz input both need to be converted from strings to numbers.
- The question file has to follow one exact format. A small formatting mistake can make a question invalid, so the loader checks the number of fields.
- The leaderboard has to be read, updated and written again every time a score is saved.
- Keeping the code simple while still separating the menu, question data and quiz logic took some trial and error.
- Testing file-based code needs temporary files, so TestQuizEngine creates and deletes its own test files.

12. Learnings & Key Takeaways

The main thing I learned from this project was how a few small Java classes can work together to make one complete program. It also helped me see why it is better to keep the menu, question data and quiz logic separate instead of writing everything inside `main()`.

File handling was another useful part. Reading a text file looks simple, but the program still has to deal with missing files, empty lines and incorrectly formatted data.

I also got more practice with input validation. A terminal program cannot assume that the user will always type a valid number, so the input has to be checked before it is used.

The testing class was useful too. It showed me that even a small project can have separate checks for individual parts instead of relying only on one manual run of the whole application.

13. Future Enhancements

- Add a timer for each question.
- Add question categories such as Java, DSA or general knowledge.
- Add difficulty levels and let the user choose how many questions to attempt.
- Try other scoring rules, such as negative marking.
- Add an admin option for adding or removing questions without editing the file manually.
- Save more information about each attempt, such as date, percentage and number of correct answers.
- Move to a database if the project becomes large enough that text files are no longer convenient.
- Make a GUI version so the same logic can be used without the terminal interface.

14. References

- Java Platform Standard Edition documentation: <https://docs.oracle.com/en/java/javase/>
- Project source code: `Main.java`, `Question.java`, `QuizManager.java` and `TestQuizEngine.java`.
- Project data files: `questions.txt` and `leaderboard.txt`.

Before submitting this report, replace each red screenshot marker with a screenshot from your own run of the program and keep the short explanation below it.